arXiv:2402.10466v4 [cs.CL] 30 May 2024

Large Language Models as Zero-shot Dialogue State Tracker through Function Calling

Zekun Li[†1], Zhiyu Zoey Chen[2], Mike Ross[3], Patrick Huber[3], Seungwhan Moon[3], Zhaojiang Lin[3], Luna Dong[3], Adithya Sagar[3], Xifeng Yan[1], and Paul A. Crook[3]

[1]University of California, Santa Barbara
[2]Carnegie Mellon University
[3]Meta AI

Abstract

Large language models (LLMs) are increasingly prevalent in conversational systems due to their advanced understanding and generative capabilities in general contexts. However, their effectiveness in task-oriented dialogues (TOD), which requires not only response generation but also effective dialogue state tracking (DST) within specific tasks and domains, remains less satisfying. In this work, we propose a novel approach FnCTOD for solving DST with LLMs through function calling. This method improves zero-shot DST, allowing adaptation to diverse domains without extensive data collection or model tuning. Our experimental results demonstrate that our approach achieves exceptional performance with both modestly sized open-source and also proprietary LLMs: with in-context prompting it enables various 7B or 13B parameter models to surpass the previous state-of-the-art (SOTA) achieved by ChatGPT, and improves ChatGPT's performance beating the SOTA by 5.6% average joint goal accuracy (JGA). Individual model results for GPT-3.5 and GPT-4 are boosted by 4.8% and 14%, respectively. We also show that by fine-tuning on a small collection of diverse task-oriented dialogues, we can equip modestly sized models, specifically a 13B parameter LLaMA2-Chat model, with function-calling capabilities and DST performance comparable to ChatGPT while maintaining their chat capabilities. We have made the code publicly available.[1]

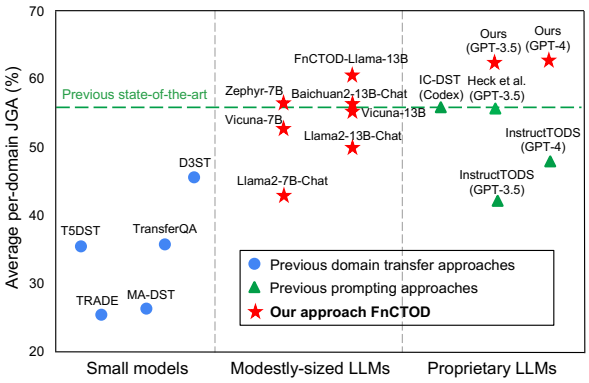

Figure 1: **Zero-shot DST performance comparison** among (1) previous domain transfer approaches using small models; (2) previous prompting approaches exclusively relying on advanced proprietary LLMs; and (3) our approach, compatible with various LLMs, empowers various 7B and 13B models for superior performance and sets new state-of-the-art with GPT-4.

1 Introduction

Recent years have seen the rapid development of large language models (LLMs) that have demonstrated exceptional natural language understanding and generation capabilities. The integration of LLMs into industry applications, particularly as conversational assistants, is a notable trend. Fine-tuned with conversations between users and assistants, these models are further aligned with human preferences to enhance their ability to deliver fluent, helpful, and polite responses to user inquiries. Notable examples include proprietary systems such as ChatGPT[2] and Claude[3], as well as open-source models such as LLaMA2-Chat (Touvron et al., 2023), Vicuna (Chiang et al., 2023), Baichuan (Baichuan, 2023).

The primary focus of these chat-tuned LLMs has typically been on responding in general contexts. However, for another important type of conversation, task-oriented dialogues (TOD), the model is required to extract the intentions of users at each turn of the conversation, represented as slot-value pairs of per-domain predefined schemas; a process known as Dialogue State Tracking (DST). The challenge lies in the model's ability to accurately summarize user needs over multiple turns of conver-

[1] https://github.com/facebookresearch/FnCTOD
†Work undertaken while interning at Meta.
*Correspondence authors: zekunli@ucsb.cs.edu, and pacrook@meta.com.

[2] http://chatgpt.openai.com/
[3] https://www.anthropic.com/index/introducing-claude

Table 1: **Comparison of different zero-shot DST paradigms.** Plug-&-Play means the (chat-tuned) LLMs can be equipped with this capability, preserving their conversational capabilities.

Zero-shot DST Paradigms	Base Model	Fine-tuning	Prompting	Plug-&-Play
Domain transfer approaches (Lin et al., 2021b,c; Zhao et al., 2022)	Small LMs	✓	✗	✗
Previous prompting approaches (Heck et al., 2023; Chung et al., 2023)	Advanced proprietary LLMs	✗	✓	✗
FnCTOD (**Ours**)	Modestly-sized open-source LLMs & Advanced proprietary LLMs	✓	✓	✓

sation and also strictly adhere to a domain-specific ontology. The most direct solutions (Hosseini-Asl et al., 2020; Peng et al., 2020; Su et al., 2021) necessitate training on curated domain-specific annotated data, a process that is notoriously costly and labor-intensive. Despite efforts in data augmentation (LI et al., 2020) and automated dataset creation using GPT-3 (Li et al., 2022), these methods struggle to generalize to unseen domains. To achieve zero-shot DST for unseen domains, prior approaches usually involved domain transfer methods (Campagna et al., 2020; Lin et al., 2021a; Zhao et al., 2022). While such approaches are trained on alternative domains, they still require domains with matching annotation schema, and their performance has been far from satisfactory.

LLMs exhibit remarkable capabilities for tackling various tasks without the need for task-specific fine-tuning, making them suited for zero-shot DST. However, while there have been initiatives to leverage ChatGPT for zero-shot DST (Hu et al., 2022; Hudeček and Dušek, 2023; Heck et al., 2023; Chung et al., 2023), these methods tend to treat DST as a standalone task rather than chat completion, which the models, especially chat-tuned models, are more proficient in. They usually take the whole conversation as input along with detailed instructions to generate in domain-specific formats. This setup poses challenges due to the long task context and specific output requirements. Consequently, this works exclusively with advanced ChatGPT or Codex models but fails with less powerful LLMs (Hudeček and Dušek, 2023).

In this work, we introduce a novel approach FnCTOD, to address zero-shot DST with LLMs. Our method seamlessly integrates DST as a part of the assistant's output during chat completion. Specifically, we treat the schema of each task-oriented dialogue domain as a specific function, and DST for this domain as the process of "calling" the corresponding function. We thus instruct LLMs to generate function calls along with the response in the assistant's output. To achieve this, we convert the domain schema into function specifications, which include the function's description and required arguments, and incorporate them into the *system prompt* of the LLM. Additionally, we integrate these function calls into the assistant's output within the *dialogue context*.

As shown in Figure 1, experimental results on the MultiWOZ benchmark (Budzianowski et al., 2018) represent a significant milestone. Our approach is the first that, without further fine-tuning, enables modestly sized open-source LLMs (7B or 13B parameters) to achieve comparable or superior performance compared to previous state-of-the-art (SOTA) prompting methods that relied exclusively on advanced proprietary LLMs such as ChatGPT and Codex (Hudeček and Dušek, 2023; Heck et al., 2023; Chung et al., 2023). Furthermore, our approach beats the previous zero-shot SOTA by 5.6% Av. JGA, firmly establishing a new standard. It improves ChatGPT performance; beating previous individual best results for GPT-3.5 and GPT-4 by 4.8% and 14%, respectively.

Additionally, we show that by fine-tuning a 13B LLaMA2-Chat model using a collection of 7,200 task-oriented dialogues — consisting of 200 randomly selected dialogues covering 36 diverse domains, from heterogeneous TOD datasets — we can equip it with function-calling DST abilities comparable to ChatGPT while still maintaining its response generation capabilities.

The comparison with prior studies is summarized in Table 1 and Figure 1. **Our contribution is threefold: (1)** Demonstration that the FnCTOD approach achieves outstanding performance with both open-source and proprietary LLMs through *in-context prompting*: enables open-source 7–13B models to surpass the previous SOTA achieved by ChatGPT, and enhances GPT-4's performance by 14%, establishing a new SOTA. **(2)** Bridging the *zero-shot* DST performance gap between open-source models and ChatGPT by fine-tuning on a small collection of diverse dialogues. **(3)** Showing that function calling DST capabilities can be

integrated into existing chat-tuned LLMs while preserving response capabilities.

2 Related Work

2.1 Dialogue State Tracking

DST is an essential, yet challenging task in the construction of TOD systems. Its primary purpose is to extract and track user goals at each turn throughout the conversation. The tracked dialogue state is usually represented in the slot values of the predefined schema for specific domains. This requires the slot values to adhere closely to the domain-specific schema. Consequently, previous methods have relied on the collection and annotation of domain-specific dialogues for model training (Lee et al., 2019; Wu et al., 2019; Heck et al., 2020; Hosseini-Asl et al., 2020; Peng et al., 2020; Lin et al., 2020) However, obtaining training data is notoriously expensive, even with data augmentation methods (LI et al., 2020) and methods that utilize GPT-3 to automatically simulate such data (Li et al., 2022). Furthermore, these approaches are limited to handling only the domains covered in the training data.

To address zero-shot DST in unseen domains, previous cross-domain transfer strategies based on small models typically leverage extra dialogue corpora in similar domains (Wu et al., 2020; Lin et al., 2021b; Su et al., 2021) or redefine DST in terms of alternative tasks, e.g., Q&A (Lin et al., 2021c) or summarization (Shin et al., 2022) to find additional data. Despite these efforts, their overall zero-shot performance remains relatively low.

2.2 Leveraging LLMs for Dialogue Tasks

LLMs (Brown et al., 2020; Chowdhery et al., 2023; OpenAI, 2023) have demonstrated remarkable capabilities in handling various tasks without further fine-tuning. Recent chat/instruction-tuned models further exhibit impressive performance in conversational contexts (Touvron et al., 2023; Chiang et al., 2023; Yang et al., 2023). However, current chat models primarily focus on general conversation, often omitting task-oriented dialogues (TOD). TOD differs from general conversation in that it requires models to not only generate responses but also track dialogue states according to domain-specific schemas. While ChatGPT has shown effectiveness in response generation for TOD tasks (Li et al., 2024), its performance in zero-shot DST, as explored in recent research on prompting approaches (Hu et al., 2022; Bang et al., 2023; Hudeček and Dušek, 2023; Heck et al., 2023; Zhang et al., 2023; Chung et al., 2023), is unsatisfactory and therefore remains as a open challenge.

2.3 Tool Use within LLMs

Early work on tool use (Parisi et al., 2022; Schick et al., 2023) and the recent launch of GPT-4 plug-in and function calling features (OpenAI, 2023), have highlighted the importance of function calling for LLMs, encouraging follow-up work (Patil et al., 2023; Shen et al., 2023; Li et al., 2023a). Commonly integrated tools include web browsers, calculators (Cobbe et al., 2021), translation systems. We are the first to utilize this tool usage/function calling capability to solve the challenging DST task in TOD with LLMs, bridging the gap between general conversation and task-oriented dialogues.

3 Background

3.1 Chat-tuned LLMs

Chat-tuned LLMs are models specifically fine-tuned to interact with users in a conversational manner. This category encompasses proprietary models such as ChatGPT and Claude, as well as open-source models such as Vicuna (Chiang et al., 2023), LLaMA2-Chat (Touvron et al., 2023), and Baichuan (Yang et al., 2023). These chat models typically start as base models that are further fine-tuned with a dialogue format, enabling them to function effectively as conversational agents. As depicted in Figure 2, the dialogue format typically features three distinct roles within two components: (1) the *system* role in the *system prompt* section, which defines the assistant's roles, responsibilities, and expected behaviors; and (2) the *user* and *assistant* roles in the *dialogue context* section, encompassing their conversation. The model is typically tasked to produce the *assistant*'s responses to the *user*'s input. These chat models are primarily designed to generate helpful, detailed, and friendly responses to general user inquiries, rather than handling task-specific conversations as in TOD.

3.2 DST Task Formulation

In TOD, at each turn of conversation, the task of DST is to summarize the dialogue state S_t given the dialogue context $C_t = \{A_1, U_1, \cdots, A_t, U_t\}$, where U_t and A_t represent the user utterance and assistant response at the t-th turn. For simplicity, we will omit the turn index t in subsequent discussions. The dialogue state S is a set of slot-value

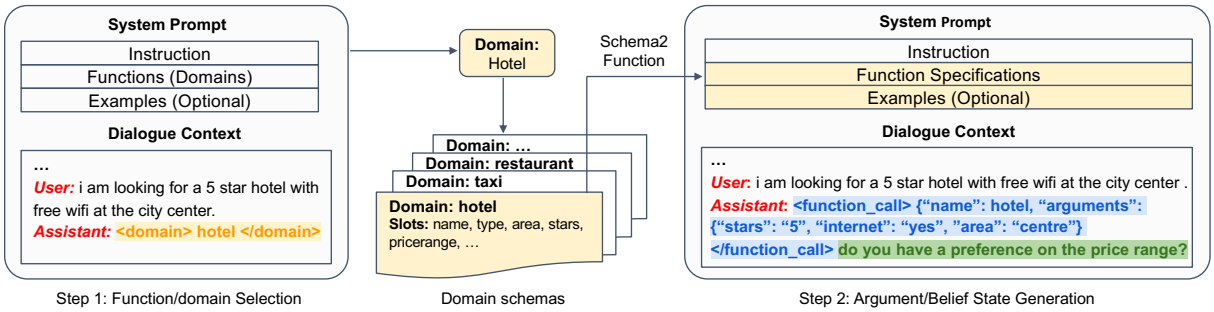

Figure 2: **Overview of our approach that addresses DST via function calling.** The whole prompt includes two sections: *system prompt* and *dialogue context*. The function calling process is executed in two stages. Initially, the model is prompted to determine the function to be called (function name). Following this, the specifications of the predicted function/domain, along with optional demonstration examples, are incorporated into the *system prompt*. This guides the model to generate function arguments and subsequently the response.

pairs:

$$S = \{(s_{1,D_1}, v_{1,D_1}), \cdots, (s_{i,D_j}, v_{i,D_j})\}, \quad (1)$$

where s_{i,D_j} is the i-th slot in the D_j domain, and v_{i,D_j} is its tracked value. Each domain D_j corresponds to a set of slots for a specific service, API call, or database query, such as restaurant reservations. In the case of the *restaurant* domain, the slots might include "restaurant-food", "restaurant-area", "restaurant-pricerange", etc. We use S_{D_j} to denote the tracked slots for domain D_j.

4 Approach

Our method redefines DST as function calling, treating each domain as a distinct function, and the slot values within the domain as its arguments. As shown in Figure 2, this paradigm is represented in chat-tuned models by embedding function specifications within system prompts, as shown in Figure 3. The model is tasked with generating function calls followed by a response, as shown in Figure 4. We provide more details of our approach below.

DST as Function Calling In our formalization, DST is conceptualized as function calling. Each domain D_j is modeled as a unique function F_j, with the associated slot values serving as arguments. Consequently, at each turn of the conversation, the DST task transforms into identifying the correct function F_j and its arguments S_{D_j}:

$$\texttt{<fn_call>}\ F_j(s_{1,D_1} = v_{1,D_1}, s_{2,D_1} = v_{2,D_1}, \cdots, s_{i,D_j} = v_{i,D_j})\ \texttt{</fn_call>}, \quad (2)$$

where "<fn_call>" and "</fn_call>" are special tokens. In practice, we use "<function_call>" and "</function_call>" to represent them, and generate the function calls in JSON format. Some examples of function calls generated within a conversation are shown in Figure 4.

```
System Prompt

You are a task-oriented assistant. You can use the given functions to
fetch further data to help the users.

<Functions>
{
  "name": "find_book_hotel",
  "description": "hotel reservations and vacation stays. ",
  "arguments": [
    {
      "name": "name",
      "type": "string",
      "description": "name of the hotel"
    },
    {
      "name": "pricerange",
      "type": "string",
      "description": "price budget of the hotel",
      "possible_values": ["expensive", "cheap", "moderate"]
    },
    ......
  ]
}
</Functions>

<Examples>
...
</Examples>
```

Figure 3: Illustration of the *system prompt* consisting of three components: (1) the overall instruction, (2) function specifications, and (3) optional example conversations. Due to space constraints, only a part of slots/arguments are displayed. The specific example conversations are omitted for brevity.

Dialogue Prompt Format As shown in Figure 4, we incorporate this function calling as an integral part of the conversation. At each turn of the conversation, given the current conversation context,

Dialogue Context

User: i would like information about museums in cambridge .

Assistant: <function_call> {"function": "find_attraction", "arguments": {"type": "museum"}} </function_call> there are 23 museums in cambridge . what part of town will you be in ?

User: can you tell me about museums in the centre of town ?

Assistant: <function_call> {"function": "find_attraction", "arguments": {"type": "museum", "area": "centre"}} </function_call> i have the castle galleries museum in the centre of town . phone number is 01223307402 .

User: i also want to find the hotel called archway house .

Assistant: <function_call> {"function": "find_book_hotel", "arguments": {"name": "archway house"}} </function_call> it is located in the north , on 52 gilbert road , cb43pe . would you like to book a room ?

User: yes , please book for 4 nights on friday for 4 people .

Assistant: <function_call> {"function": "find_book_hotel", "arguments": {"name": "archway house", "area": "north", "people": "4", "day": "friday", "stay": "4"}} </function_call> all set , the reference number is 6s9gcfdo , can i help with anything else ?

User: that is all . thank you so much for your help .

Assistant: have a great day ! goodbye !

Figure 4: An example of the *dialogue context* including attraction and hotel domains. The assistant output consists of the function calls and response .

the chat model is tasked with first generating the function call, followed by the response. To achieve that, we convert the domain schema into function specifications, using JSON, and include them in the *system prompt* within the dialogue prompt, as shown in Figure 3. By doing so, the model can be aware of the supported functions and the correct generation format to call the function.

Function Call Decomposition As outlined, the model is tasked with predicting not only the function to call (*i.e.*, function name) but also generating arguments for the predicted function. Providing detailed specifications for all the supported functions would make the prompt very lengthy. To streamline this process and minimize the prompt length, we split the whole generation process into two consecutive steps: **Function Selection** and **Argument Generation**. As shown in Figure 2, during each turn of the conversation, the model first selects a function F_j from all supported functions. In this step, we provide concise function descriptions rather than detailed specifications in the *system prompt* and instruct the model to generate only the selected domain/function, surrounded by the special tokens "<domain>" and "</domain>". Subsequently, we include the detailed specification of only the chosen function F_j in the *system prompt*, prompting the model to generate the corresponding arguments F_j. This approach not only simplifies the task but also significantly reduces the prompt length, enhancing both efficacy and efficiency.

In-context Prompting Since the current open-source models are not specifically fine-tuned to generate function calls, there is no guarantee that the model could always generate the correct formats. To address that, we also include in-context example conversations as shown in Figure 4, along with the specification of the predicted function in the *system prompt*. We manually selected a few demonstration examples for each supported domain.

Prompt-based Fine-tuning To illustrate equipping an open source model with function calling capabilities, eliminating the need for demonstration examples, we fine-tune a LLAMA2-13B-CHAT model using a collection of heterogeneous task-oriented dialogue datasets, including WOZ (Mrkšić et al., 2016), CamRest676 (Wen et al., 2016b,a), MSR-E2E (Li et al., 2018), TaskMaster (Byrne et al., 2019) and Schema-Guided Dialogues (SGD) (Rastogi et al., 2020). Note that we deliberately exclude the entire target dataset. From these datasets, we choose 36 distinct domains with high-quality annotations. Instead of using all the data in those datasets, we randomly sampled 200 dialogues from each domain across the datasets, totaling 7,200 dialogues for training. This small sample size proved sufficient.

We convert these dialogues into our designed dialogue prompt. Specifically, we incorporate the specifications of all functions invoked in each conversation's *system prompt*. Our loss calculation focused solely on the function calling aspect of the assistant's generation. We refrained from fine-tuning the response generation component, in consideration of the LLMs' existing competence in producing coherent responses and the scarcity of function-calling examples in our dataset. Our fine-tuned model is dubbed FNCTOD-LLAMA2-13B.

5 Experiments

5.1 Experimental Setup

Dataset and Metrics We evaluate on the widely-used task-oriented multi-domain dataset **MultiWOZ 2.1** (Budzianowski et al., 2018; Eric et al., 2020). We used the 1,000 dialogues in the test split and measured joint goal accuracy (JGA), which measures the percentage of turns for which all slot values are correctly predicted. This test set spans 5

Model	Attraction		Hotel		Restaurant		Taxi		Train		JGA	
	JGA	Slot-F1	JGA	Slot-F1	JGA	Slot-F1	JGA	Slot-F1	JGA	Slot-F1	Average	Overall
Cross-domain Transfer approaches												
TRADE (Wu et al., 2019)	20.06	–	14.20	–	12.59	–	59.21	–	22.39	–	25.69	–
MA-DST (Kumar et al., 2020)	22.46	–	16.28	–	13.56	–	59.27	–	22.76	–	26.87	–
TransferQA (Lin et al., 2021b)	31.25	–	22.72	–	26.28	–	61.87	–	36.72	–	35.77	–
T5DST (Lin et al., 2021c)	33.09	–	21.21	–	21.65	–	64.62	–	35.43	–	35.20	–
D3ST (Zhao et al., 2022)	56.40	–	21.80	–	38.20	–	**78.40**	–	38.70	–	46.70	–
Previous Prompting approaches												
*IC-DST (Codex)	60.00	–	46.70	–	57.30	–	71.40	–	49.40	–	56.96	–
Heck et al. (2023) (GPT-3.5)	52.70	–	42.00	–	55.80	–	70.90	–	60.80	–	56.44	31.50
InstructTODS (GPT-3.5)	30.23	65.38	26.77	76.28	48.28	82.90	56.22	75.33	53.75	83.64	42.02	–
InstructTODS (GPT-4)	39.53	78.99	31.23	84.07	55.86	88.23	63.24	82.71	59.83	89.72	48.16	–
***Our approach* FNCTOD**												
ChatGPT (GPT-3.5)	**67.15**	**87.20**	37.56	82.86	60.12	90.21	74.43	86.90	67.29	**92.48**	61.31	38.56
ChatGPT (GPT-4)	58.77	81.84	45.15	85.07	63.18	91.06	76.39	**87.73**	**69.48**	90.16	**62.59**	**38.71**
FNCTOD-LLAMA2-13B	62.24	84.99	**46.83**	**85.39**	60.27	88.69	67.48	80.39	60.90	89.88	59.54	37.67
ZEPHYR-7B-BETA	56.50	81.97	38.43	79.52	**63.18**	**91.19**	74.10	86.56	56.20	90.00	57.68	32.11
VICUNA-7B-V1.5	50.66	74.93	35.00	73.66	52.76	85.25	67.02	80.36	59.66	89.05	53.02	29.45
VICUNA-13B-V1.5	54.25	80.99	38.43	79.96	56.44	87.26	69.11	83.37	58.82	89.26	55.41	31.84
BAICHUAN2-13B-CHAT	53.67	79.57	40.15	81.36	59.02	87.82	69.31	81.95	60.67	89.45	56.56	33.21
LLAMA2-7B-CHAT	42.64	70.18	30.47	69.37	37.60	78.63	63.20	73.80	44.17	82.18	43.44	16.78
LLAMA2-13B-CHAT	49.76	76.80	29.50	67.60	48.87	81.33	64.66	68.97	53.59	85.09	49.28	25.68
LLAMA2-70B-CHAT	50.66	78.26	34.03	76.61	54.48	86.18	66.10	72.60	56.53	87.39	52.36	28.38

Table 2: **Performance comparison on zero-shot DST benchmark**. We compare our approach with cross-domain approaches and prompting approaches relying on ChatGPT (GPT-3.5/4) and Codex. Using our approach, we evaluate ChatGPT, and our fine-tuned model via zero-shot prompting, and open-source models via few-shot (5-shot) prompting. In addition to per-domain JGA and slot F1 scores, we report the macro-averaged JGA of these five domains (**Average JGA**), and also the multi-domain JGA (**Overall JGA**). The baseline results are directly taken from their respective works. The best performances in each column are in **bold**.

domains, with each conversation potentially covering multiple domains.

Baselines We compare our approach with two distinct approaches: (1) *Cross-domain transfer approaches*, which involve training on MultiWOZ with one domain excluded and then evaluating on the held-out domain. This category includes TRADE (Wu et al., 2019), MA-DST (Kumar et al., 2020), TransferQA (Lin et al., 2021b), T5DST (Lin et al., 2021c), and D3ST (Zhao et al., 2022). (2) *Previous prompting approaches* that have only shown efficacy with advanced proprietary models, include IC-DST (Hu et al., 2022) using Codex, (Heck et al., 2023) and InstructTODS (Chung et al., 2023) using ChatGPT (GPT-3.5/4).

Evaluated Models We evaluate our method on proprietary ChatGPT and various open-source models. For ChatGPT, we evaluated the versions of GPT-3.5-Turbo (`gpt-3.5-turbo-1106`) and GPT-4 (`gpt-4-1106-preview`), both of which are already equipped with function calling capabilities. Regarding open-source models, we assessed several widely recognized chat-tuned models of varying sizes, including the 7B parameter model ZEPHYR-7B-BETA (Tunstall et al., 2023), the 7B and 13B versions of VICUNA-V1.5 (Chiang et al., 2023), the 7B, 13B, and 70B versions of LLAMA2-CHAT (Touvron et al., 2023), as well as the 13B parameter model BAICHUAN2-13B-CHAT (Baichuan, 2023).

Additionally, we evaluate our fine-tuned model FNCTOD-LLAMA2-13B. It's worth noting that unlike these domain transfer baselines, our model is trained exclusively on 7,200 dialogues from datasets other than MultiWOZ, making the setup more realistic and challenging.

Inference Details For both ChatGPT and our fine-tuned FNCTOD-LLAMA2-13B, which have been equipped with function-calling capabilities, we perform zero-shot prompting, excluding in-context examples in the system prompt. For the other open-source models, we perform few-shot prompting using five examples (5-shot) by default. It's worth noting that the shot in zero/few-shot prompting refers to the number of in-context examples used when prompting the models, whereas the shot in zero-shot DST refers to the number of in-domain examples seen in the training data.

5.2 Zero-shot DST Evaluation

Table 2 presents the zero-shot DST performance comparison, with observations summarized below.

*IC-DST requires in-domain data to train the retriever for example selection, making it not strictly zero-shot DST.

Our approach empowers moderately-sized open-source models to surpass previous SOTA results achieved with advanced ChatGPT. Previous prompting approaches showed promising results exclusively with advanced proprietary models but underperformed with less advanced models (Hudeček and Dušek, 2023). Our approach is the first to enable moderately sized open-source models to achieve comparable or superior performance compared to previous SOTA results obtained with advanced ChatGPT and Codex. Specifically, the 7B parameter ZEPHYR-7B-BETA and 13B parameter BAICHUAN2-13B-CHAT models outperform the previous SOTA. This significant advancement marks a milestone in the practical application of LLMs for DST and TOD.

Our approach significantly improves ChatGPT's performance over previous prompting approaches. The efficacy of our approach is verified by improvements of 4.8% (Average JGA) for GPT-3.5, and 14% for GPT-4, compared to previous reported results with each of these models. Our result with GPT-4 beats the previous SOTA prompting approach using Codex by 5.6% Avergage JGA.

Our fine-tuned 13B parameter model matches the performance of ChatGPT. It is evident that our fine-tuned FNCTOD-LLAMA2-13B significantly improves over its base model LLAMA2-13B-CHAT and achieves a performance comparable to ChatGPT. This demonstrates that we can easily equip moderately sized open-source LLMs with function-calling capabilities and zero-shot DST performance comparable to ChatGPT, marking an exciting advance in bridging the gap between open-source and proprietary models.

5.3 Zero-shot End-to-End TOD Evaluation

In practical settings, a TOD system queries a knowledge base or API using the tracked dialogue states to ground responses. We perform an end-to-end evaluation of both DST and response generation, which is a more realistic and challenging setting. Our FNCTOD approach enables the generation of both dialogue states, *i.e.*, function calls, *and* responses in the assistant's output. This contrasts with the prompting methods that typically treat DST as a standalone task. Consistent with the previous work on end-to-end zero-shot TOD evaluation (Hudeček and Dušek, 2023), we evaluated using the MultiWOZ 2.2 dataset (Zang et al., 2020) with delexicalized responses. Our evaluation metrics include JGA for DST and *Success* rate for the generated response. Success measures the percentage of dialogues in which the user's goals were fully met (Budzianowski et al., 2018). The results are presented in Table 3.

Model	JGA	Success
ChatGPT (Hudeček and Dušek, 2023)	27.0	44.0
FNCTOD-LLAMA2-13B	**37.9**	44.4
ZEPHYR-7B-BETA	32.3	**57.5**
VICUNA-7B-V1.5	29.4	37.7
VICUNA-13B-V1.5	33.8	23.1
BAICHUAN2-13B-CHAT	33.0	45.7
LLAMA2-7B-CHAT	16.7	24.9
LLAMA2-13B-CHAT	25.8	27.7

Table 3: End-to-end evaluation results on MultiWOZ 2.2, including the evaluation on DST with (overall) **JGA** and also response generation with **Success rate**.

Compared to previous prompting approaches, by enabling both zero-shot DST and response generation (Hudeček and Dušek, 2023), the superiority of the FnCTOD approach becomes more evident. Specifically, all open-source models evaluated using our approach outperform ChatGPT's results achieved by (Hudeček and Dušek, 2023), except for LLAMA2-7B-CHAT. In addition, the results show that the fine-tuned model FNCTOD-LLAMA2-13B retains its ability to generalize and generate informative responses in a zero-shot TOD setting.

5.4 Ablation Studies

Impact of function call decomposition Our two-step decomposition approach can yield benefits in terms of both efficacy and efficiency. To demonstrate this, we compared the accuracy and cost with and without the two-step decomposition. The "without decomposition" condition includes the full specification of all domains/functions in the prompt, and requires the LLM to generate the full function call, including the function name and arguments in one step. This comparison is conducted on ChatGPT and our fine-tuned FNCTOD-LLAMA2-13B, which supports zero-shot prompting. Since ChatGPT is API-accessible, we compare its API calling cost which is calculated based on the number of tokens in the prompt. The results in Table 4 demonstrate both the effectiveness and efficiency of our decomposition approach.

In addition, to investigate the impact of cascading, two-stage errors due to incorrect function (name) prediction, we conduct experiments to com-

Method	JGA (↑)						Tokens (m) (↓)			Time (h) (↓)			API (↓)
	Attr.	Hotel	Rest.	Taxi	Train	Overall	FS	AG	Total	FS	AG	Total	Cost ($)
ChatGPT (GPT-3.5)													
Non-decomp.	59.64	32.24	61.39	74.87	49.91	30.16	-	-	-	-	-	-	13.32
Decomposition	**67.15**	**37.56**	60.12	74.43	**67.29**	**38.56**	-	-	-	-	-	-	**8.97**
FNCTOD-LLAMA2-13B													
Non-decomp.	34.77	32.02	56.63	65.40	36.21	21.04	-	-	23.57	-	-	12.53	-
Decomposition	**62.24**	**46.83**	**60.27**	**67.48**	**60.90**	**37.67**	5.24	7.77	**13.01**	0.80	8.64	**9.44**	-

Table 4: Ablation studies on the function call generation decomposition, where decomp. denotes decomposition. The comparison on efficiency includes the API cost of GPT-3.5 and local inference time and consumed tokens for FNCTOD-LLAMA2-13B on a single Nvidia A6000 GPU, where FS stands for Function Selection (the first step) and AG stands for Argument Generation (the second step). Best results are bolded.

Model	Oracle	FS Acc.	JGA
ChatGPT (GPT-4)	×	88.62	38.71
ChatGPT (GPT-4)	✓	-	44.44
ChatGPT (GPT-3.5)	×	95.54	38.56
ChatGPT (GPT-3.5)	✓	-	38.32
FNCTOD-LLAMA2-13B	×	91.68	37.67
FNCTOD-LLAMA2-13B	✓	-	37.93
ZEPHYR-7B-BETA	×	92.77	32.11
ZEPHYR-7B-BETA	✓	-	34.40
VICUNA-7B-V1.5	×	94.75	29.45
VICUNA-7B-V1.5	✓	-	30.06
VICUNA-13B-V1.5	×	91.82	31.84
VICUNA-13B-V1.5	✓	-	34.20
BAICHUAN2-13B-CHAT	×	92.50	33.21
BAICHUAN2-13B-CHAT	✓	-	34.93
LLAMA2-7B-CHAT	×	91.90	16.78
LLAMA2-7B-CHAT	✓	-	18.75
LLAMA2-13B-CHAT	×	89.34	25.68
LLAMA2-13B-CHAT	✓	-	26.56

Table 5: Model performances with oracle and predicted functions/domains. FS Acc. indicates the function selection accuracy, and the JGA here indicates overall JGA.

pare the performance with oracle and predicted functions. The results are shown in Table 5. As can be seen, while errors do impact the overall performance, the significance depends on the prediction accuracy. Many LLMs, especially ChatGPT (GPT-3.5), demonstrate very high prediction accuracy considering potential label noise. Performance with oracle and predicted functions are largely comparable. Improving the function prediction accuracy could further boost the performance of our approach.

Impact of function specifications In addition to directly including function specifications in JSON within the prompt, we experimented with translating the data into more human-readable natural language descriptions (refer to the comparison in Figure 7 in the Appendix). Figure 5 presents a comparison between using the JSON format directly (json) and converting it into natural language descriptions (text). The results indicate that the models perform similarly with both methods of function specification, indicating the high degree of flexibility of our approach in function specification customization.

Impact of the unified dialogue prompt In our approach, we seamlessly integrated function calls into the assistant's output in the conversation context, which could also serve as demonstration for the current turn's generation. To investigate its effect, we show the performance with and without function calls (w/ and w/o prev) in Figure 5. The results emphasize the effectiveness of embedding function calls within the conversation context.

Impact of varying numbers of in-context examples We assessed the performance of various open-source models, which were not originally trained for function call generation, with different numbers of in-context examples, ranging from 0 to 5. We note that using more than five examples might surpass the context-window capacity (such as 4096 tokens) for some models. The findings are illustrated in Figure 6. The results indicate that the models perform significantly better when in-context examples are utilized compared to zero-shot prompting. There is a consistent performance improvement as the number of examples increases, across most domains and models.

Impact of fine-tuning data sizes Our results indicate that with as few as 200 samples per domain, totaling 7,200 dialogues across 36 domains, we were able to fine-tune a LLAMA2-13B-CHAT model to match the zero-shot DST performance of ChatGPT. We explored the model's performance with varying numbers of samples, ranging from 100 to 400 per domain. The results, depicted in

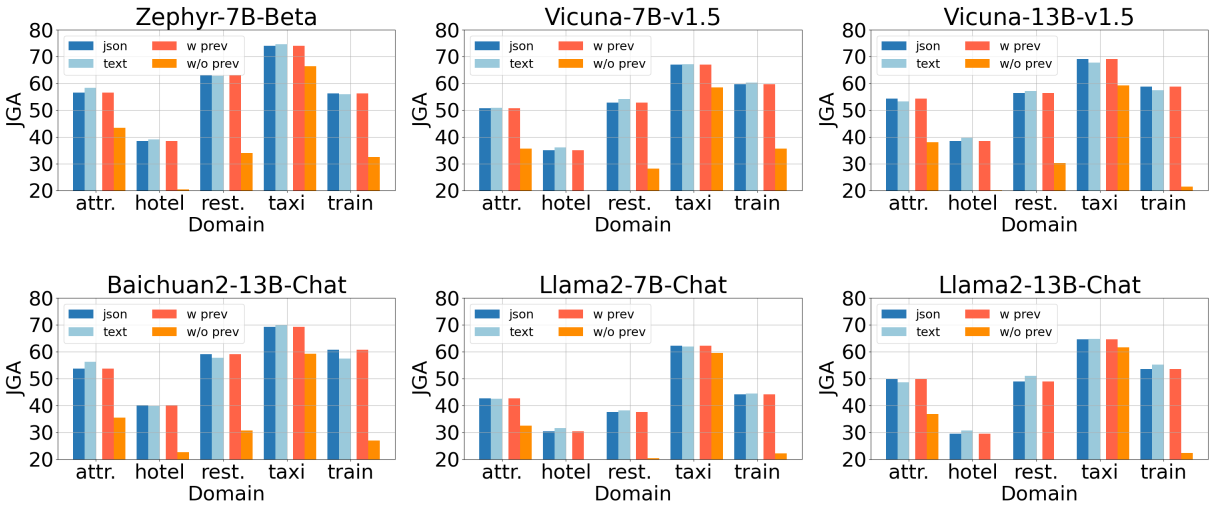

Figure 5: Ablation studies on different function specification types (json/text) and the unified dialogue format including or not including function calls in previous conversation context (w/ and w/o prev).

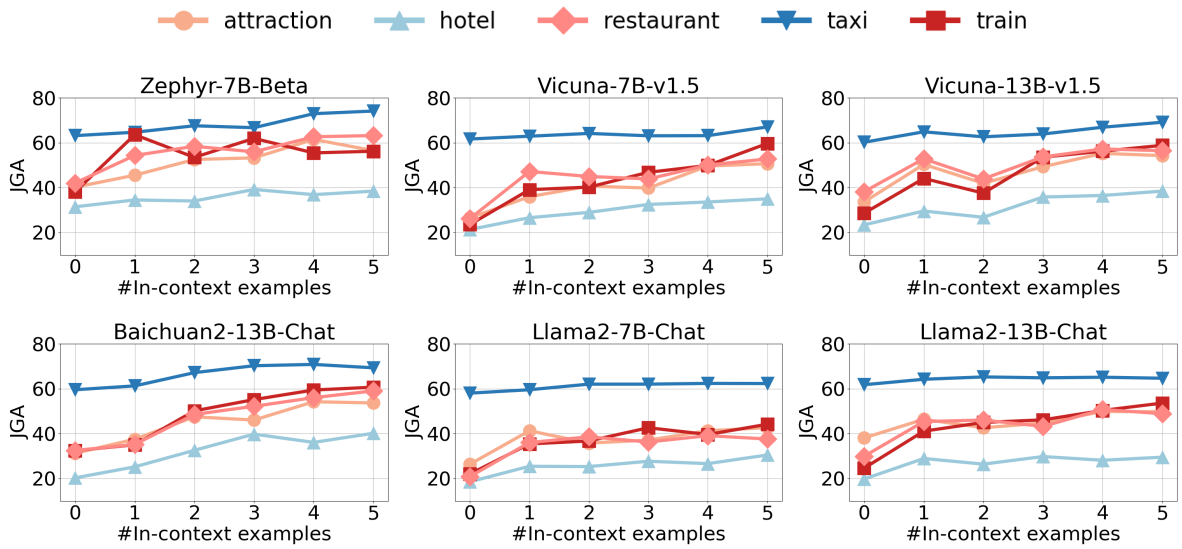

Figure 6: Performance of open-source models with different numbers of in-context examples.

Table 6, show that optimal performance is achieved with 200 samples per domain. We speculate that beyond this point, the number of training samples leads to the model over-fitting to domains in the training data and, therefore, less effective at zero-shot generalization. However, we anticipate that increasing the data size through increased diversity of datasets would lead to further improvements.

#Data	Attr.	Hotel	Rest.	Taxi	Train	Avg.
100	59.61	44.40	54.33	67.02	54.33	55.94
200	62.24	**46.83**	**60.27**	**67.48**	**60.90**	**59.54**
300	**69.19**	43.68	57.06	64.98	57.60	58.50
400	60.80	43.21	57.39	65.70	53.78	56.18

Table 6: FNCTOD-LLAMA2-13B with varying numbers of training data per domain (36 domains in total).

6 Conclusion

We introduce a new approach to tackle the challenging task of zero-shot DST with LLMs, enabling them to handle both general conversations and task-oriented dialogues in diverse domains without the need for additional data collection. Our experimental results on MultiWOZ demonstrate that our approach not only delivers exceptional performance in advanced ChatGPT models (setting a new benchmark) but also across a range of moderately sized open-source LLMs. Furthermore, we demonstrate that we can fine-tune the open-source model LLAMA-2-13B-CHAT using only 7,200 training samples from 36 diverse domains, resulting in FNCTOD-LLAMA2-13B, which achieves function calling, zero-shot DST performance comparable to ChatGPT.

7 Limitations

In this work, we propose a novel approach to solve zero-shot DST with LLMs. Our approach achieves outstanding performance with various LLMs, both modestly-sized open-source and advanced proprietary LLMs, setting the new state-of-the-art. However, it is important to recognize that the current accuracy may still not be good enough for the practical deployment of such zero-shot systems. We anticipate that with further advancements in the NLU and NLG capabilities of base LLMs, our approach could achieve even greater performance levels. In addition, our approach can handle both the DST and response generation task in TOD. We evaluate DST with the well-established metric JGA with results suggesting the strong zero-shot DST performance of our approach. For the response evaluation, due to the current lack of a more realistic evaluation setting for response generation in TOD, we evaluated delexicalized responses as this is widely used in prior work. This setting and associated metrics have some known shortfalls in terms of being able to game-the-metrics with nonnatural responses as well as presenting a data mismatch with how LLMs are trained. In the era of LLMs, we advocate for the development of more realistic evaluation approaches for full-natural-language-response generation in TOD. Additionally, while this work concentrates DST and response generation, the two critical tasks in TOD, our approach can also be extended to include other complex tasks and handle various scenarios in dialogues as in the LangChain framework. We plan to explore these extensions in future research.

References

Baichuan. 2023. Baichuan 2: Open large-scale language models. *arXiv preprint arXiv:2309.10305*.

Yejin Bang, Samuel Cahyawijaya, Nayeon Lee, Wenliang Dai, Dan Su, Bryan Wilie, Holy Lovenia, Ziwei Ji, Tiezheng Yu, Willy Chung, et al. 2023. A multitask, multilingual, multimodal evaluation of chatgpt on reasoning, hallucination, and interactivity. *arXiv preprint arXiv:2302.04023*.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.

Paweł Budzianowski, Tsung-Hsien Wen, Bo-Hsiang Tseng, Iñigo Casanueva, Stefan Ultes, Osman Ramadan, and Milica Gašić. 2018. MultiWOZ - a large-scale multi-domain Wizard-of-Oz dataset for task-oriented dialogue modelling. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 5016–5026, Brussels, Belgium. Association for Computational Linguistics.

Bill Byrne, Karthik Krishnamoorthi, Chinnadhurai Sankar, Arvind Neelakantan, Daniel Duckworth, Semih Yavuz, Ben Goodrich, Amit Dubey, Andy Cedilnik, and Kyu-Young Kim. 2019. Taskmaster-1: Toward a realistic and diverse dialog dataset. *arXiv preprint arXiv:1909.05358*.

Giovanni Campagna, Agata Foryciarz, Mehrad Moradshahi, and Monica S Lam. 2020. Zero-shot transfer learning with synthesized data for multi-domain dialogue state tracking. *arXiv preprint arXiv:2005.00891*.

Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113.

Willy Chung, Samuel Cahyawijaya, Bryan Wilie, Holy Lovenia, and Pascale Fung. 2023. Instructtods: Large language models for end-to-end task-oriented dialogue systems. *arXiv preprint arXiv:2310.08885*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Mihail Eric, Rahul Goel, Shachi Paul, Abhishek Sethi, Sanchit Agarwal, Shuyang Gao, Adarsh Kumar, Anuj Goyal, Peter Ku, and Dilek Hakkani-Tur. 2020. MultiWOZ 2.1: A consolidated multi-domain dialogue dataset with state corrections and state tracking baselines. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 422–428, Marseille, France. European Language Resources Association.

Michael Heck, Nurul Lubis, Benjamin Ruppik, Renato Vukovic, Shutong Feng, Christian Geishauser, Hsien-Chin Lin, Carel van Niekerk, and Milica Gašić. 2023. Chatgpt for zero-shot dialogue state tracking: A solution or an opportunity? *arXiv preprint arXiv:2306.01386*.

Michael Heck, Carel van Niekerk, Nurul Lubis, Christian Geishauser, Hsien-Chin Lin, Marco Moresi, and Milica Gašić. 2020. Trippy: A triple copy strategy for value independent neural dialog state tracking. *arXiv preprint arXiv:2005.02877*.

Ehsan Hosseini-Asl, Bryan McCann, Chien-Sheng Wu, Semih Yavuz, and Richard Socher. 2020. A simple language model for task-oriented dialogue. *Advances in Neural Information Processing Systems*, 33:20179–20191.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

Yushi Hu, Chia-Hsuan Lee, Tianbao Xie, Tao Yu, Noah A Smith, and Mari Ostendorf. 2022. In-context learning for few-shot dialogue state tracking. *arXiv preprint arXiv:2203.08568*.

Vojtěch Hudeček and Ondřej Dušek. 2023. Are llms all you need for task-oriented dialogue? *arXiv preprint arXiv:2304.06556*.

Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7b. *arXiv preprint arXiv:2310.06825*.

Adarsh Kumar, Peter Ku, Anuj Goyal, Angeliki Metallinou, and Dilek Hakkani-Tur. 2020. Ma-dst: Multi-attention-based scalable dialog state tracking. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 8107–8114.

Hwaran Lee, Jinsik Lee, and Tae-Yoon Kim. 2019. Sumbt: Slot-utterance matching for universal and scalable belief tracking. *arXiv preprint arXiv:1907.07421*.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023a. Api-bank: A comprehensive benchmark for tool-augmented llms. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3102–3116.

SHIYANG LI, Semih Yavuz, Kazuma Hashimoto, Jia Li, Tong Niu, Nazneen Rajani, Xifeng Yan, Yingbo Zhou, and Caiming Xiong. 2020. Coco: Controllable counterfactuals for evaluating dialogue state trackers. In *International Conference on Learning Representations*.

Xiujun Li, Sarah Panda, Jingjing Liu, and Jianfeng Gao. 2018. Microsoft dialogue challenge: Building end-to-end task-completion dialogue systems. *arXiv preprint arXiv:1807.11125*.

Xuechen Li, Tianyi Zhang, Yann Dubois, Rohan Taori, Ishaan Gulrajani, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023b. Alpacaeval: An automatic evaluator of instruction-following models. https://github.com/tatsu-lab/alpaca_eval.

Zekun Li, Wenhu Chen, Shiyang Li, Hong Wang, Jing Qian, and Xifeng Yan. 2022. Controllable dialogue simulation with in-context learning. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 4330–4347.

Zekun Li, Baolin Peng, Pengcheng He, Michel Galley, Jianfeng Gao, and Xifeng Yan. 2024. Guiding large language models via directional stimulus prompting. *Advances in Neural Information Processing Systems*, 36.

Zhaojiang Lin, Bing Liu, Andrea Madotto, Seungwhan Moon, Paul Crook, Zhenpeng Zhou, Zhiguang Wang, Zhou Yu, Eunjoon Cho, Rajen Subba, et al. 2021a. Zero-shot dialogue state tracking via cross-task transfer. *arXiv preprint arXiv:2109.04655*.

Zhaojiang Lin, Bing Liu, Andrea Madotto, Seungwhan Moon, Zhenpeng Zhou, Paul Crook, Zhiguang Wang, Zhou Yu, Eunjoon Cho, Rajen Subba, and Pascale Fung. 2021b. Zero-shot dialogue state tracking via cross-task transfer. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 7890–7900, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Zhaojiang Lin, Bing Liu, Seungwhan Moon, Paul Crook, Zhenpeng Zhou, Zhiguang Wang, Zhou Yu, Andrea Madotto, Eunjoon Cho, and Rajen Subba. 2021c. Leveraging slot descriptions for zero-shot cross-domain dialogue StateTracking. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5640–5648, Online. Association for Computational Linguistics.

Zhaojiang Lin, Andrea Madotto, Genta Indra Winata, and Pascale Fung. 2020. Mintl: Minimalist transfer learning for task-oriented dialogue systems. *arXiv preprint arXiv:2009.12005*.

Nikola Mrkšić, Diarmuid O Séaghdha, Tsung-Hsien Wen, Blaise Thomson, and Steve Young. 2016. Neural belief tracker: Data-driven dialogue state tracking. *arXiv preprint arXiv:1606.03777*.

OpenAI. 2023. Gpt-4 technical report.

Aaron Parisi, Yao Zhao, and Noah Fiedel. 2022. Talm: Tool augmented language models. *arXiv preprint arXiv:2205.12255*.

Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2023. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334*.

Baolin Peng, Chunyuan Li, Jinchao Li, Shahin Shayandeh, Lars Liden, and Jianfeng Gao. 2020. Soloist:

Few-shot task-oriented dialog with a single pre-trained auto-regressive model. *arXiv preprint arXiv:2005.05298*, 3.

Abhinav Rastogi, Xiaoxue Zang, Srinivas Sunkara, Raghav Gupta, and Pranav Khaitan. 2020. Towards scalable multi-domain conversational agents: The schema-guided dialogue dataset. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 8689–8696.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*.

Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. Hugginggpt: Solving ai tasks with chatgpt and its friends in huggingface. *arXiv preprint arXiv:2303.17580*.

Jamin Shin, Hangyeol Yu, Hyeongdon Moon, Andrea Madotto, and Juneyoung Park. 2022. Dialogue summaries as dialogue states (DS2), template-guided summarization for few-shot dialogue state tracking. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 3824–3846, Dublin, Ireland. Association for Computational Linguistics.

Yixuan Su, Lei Shu, Elman Mansimov, Arshit Gupta, Deng Cai, Yi-An Lai, and Yi Zhang. 2021. Multi-task pre-training for plug-and-play task-oriented dialogue system. *arXiv preprint arXiv:2109.14739*.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

Lewis Tunstall, Edward Beeching, Nathan Lambert, Nazneen Rajani, Kashif Rasul, Younes Belkada, Shengyi Huang, Leandro von Werra, Clémentine Fourrier, Nathan Habib, et al. 2023. Zephyr: Direct distillation of lm alignment. *arXiv preprint arXiv:2310.16944*.

Tsung-Hsien Wen, Milica Gašić, Nikola Mrkšić, Lina M. Rojas-Barahona, Pei-Hao Su, Stefan Ultes, David Vandyke, and Steve Young. 2016a. Conditional generation and snapshot learning in neural dialogue systems. *arXiv preprint: 1606.03352*.

Tsung-Hsien Wen, David Vandyke, Nikola Mrkšić, Milica Gašić, Lina M. Rojas-Barahona, Pei-Hao Su, Stefan Ultes, and Steve Young. 2016b. A network-based end-to-end trainable task-oriented dialogue system. *arXiv preprint: 1604.04562*.

Chien-Sheng Wu, Steven C.H. Hoi, and Caiming Xiong. 2020. Improving limited labeled dialogue state tracking with self-supervision. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4462–4472, Online. Association for Computational Linguistics.

Chien-Sheng Wu, Andrea Madotto, Ehsan Hosseini-Asl, Caiming Xiong, Richard Socher, and Pascale Fung. 2019. Transferable multi-domain state generator for task-oriented dialogue systems. *arXiv preprint arXiv:1905.08743*.

Aiyuan Yang, Bin Xiao, Bingning Wang, Borong Zhang, Chao Yin, Chenxu Lv, Da Pan, Dian Wang, Dong Yan, Fan Yang, et al. 2023. Baichuan 2: Open large-scale language models. *arXiv preprint arXiv:2309.10305*.

Xiaoxue Zang, Abhinav Rastogi, Srinivas Sunkara, Raghav Gupta, Jianguo Zhang, and Jindong Chen. 2020. Multiwoz 2.2: A dialogue dataset with additional annotation corrections and state tracking baselines. *arXiv preprint arXiv:2007.12720*.

Xiaoying Zhang, Baolin Peng, Kun Li, Jingyan Zhou, and Helen Meng. 2023. Sgp-tod: Building task bots effortlessly via schema-guided llm prompting. *arXiv preprint arXiv:2305.09067*.

Jeffrey Zhao, Raghav Gupta, Yuan Cao, Dian Yu, Mingqiu Wang, Harrison Lee, Abhinav Rastogi, Izhak Shafran, and Yonghui Wu. 2022. Description-driven task-oriented dialog modeling. *arXiv preprint arXiv:2201.08904*.

A Appendix

A.1 Evaluation Details

Model and Inference Details We evaluated two versions of ChatGPT and six leading chat/instruction-tuned LLMs representing varying sizes and instruction-following and conversational capabilities. The six evaluated open-source models include: ZEPHYR-7B-BETA (Tunstall et al., 2023) is an instruction-tuned version of Mistral-7B (Jiang et al., 2023), which is the leading model among its size on the AlpacaEval leaderboard (Li et al., 2023b). VICUNA-7B-V1.5 and VICUNA-13B-V1.5 (Chiang et al., 2023) are LLAMA-2 models fine-tuned on user conversations with ChatGPT. LLAMA2-7B-CHAT and LLAMA2-13B-CHAT are chat-tuned versions of LLAMA2 models with varying sizes (Touvron et al., 2023). BAICHUAN2-13B-CHAT is also a LLAMA2-13B model further fine-tuned on extensive corpus (Baichuan, 2023). We utilized checkpoints available on Huggingface[4]. The specific paths for these models are detailed in Table 8. For inference, the temperature was fixed as 0.3, top_p as 0.2, and max_tokens as 128. For each test case, we conducted a single inference run. All inferences were executed on a cluster equipped with eight 48G NVIDIA RTX A6000 GPUs.

End-to-end Evaluation Setup Our end-to-end evaluation of task-oriented dialogues follows well-established standards in the literature. The traditional TOD computational process contains multiple steps, of which this work focuses on two: (1) dialogue state tracking (DST) which tracks the mentioned slots in order to query a back-end knowledge source, e.g., database or API, and (2) response generation (NLG) wherein the model generates a response given the context and query results. Following the literature, our end-to-end evaluation includes the evaluation of DST with JGA (our main results) *at the turn level*, and the evaluation of generated responses (NLG) using "Success rate" *at the dialogue level*. The latter measures the percentage of dialogues in which the user's goals were fully met. We do not use or evaluate dialog acts, as they are not necessary for response generation.

The responses of the TOD system should provide specific information about a set of entities as requested by the user, like a restaurant or hotel. As multiple entities may satisfy the user's goal and it can be difficult to detect whether the response relates to a matching entity, delexicalized responses have been used in previous literature to allow for easier measurement and to decouple NLG errors from retrieval errors. In a delexicalized response, the mentioned slot values are noted as [value_xxx]. For instance, a delexicalized response might be: "It is a restaurant that serves [value_food] food near the [value_area]. The address is [value_address] and the phone number is [value_phone]." This strategy facilitates easier automatic determination of whether the dialogue answers specific requests by the conversation's end. If both, all entities of the type expected are mentioned, and provision of all required information about those entities is also detected, then the dialogue is considered successful (Budzianowski et al., 2018). This results in a binary "Success" metric per dialogue. Regardless of the user's input, the evaluation is on whether the entities and information about those entities provided to the user by the end of the conversation satisfies their goal. For example, given a user goal at the beginning of a dialogue to obtain the phone number of a restaurant serving French cuisine, the dialogue is marked as a success if by the end the system has mentioned at least one restaurant and provided a restaurant phone number. While JGA indicates whether the correct search for a restaurant serving French food was attempted.

A.2 Training Details

Hyperparameter	Values
batch size	8
epochs	1
learning rate	0.0003
learning rate scheduler	cosine
weight decay	0.01
cutoff_len	4096
lora_r	16
lora_alpha	16
lora_dropout	0.05
lora_target_modules	q_proj, v_proj

Table 7: Hyperparameters for the model fine-tuning.

Training Data For constructing our fine-tuning dataset, we selected five high-quality, multi-turn TOD corpora, excluding MultiWOZ, as detailed in Table 9. Each dataset encompasses one or multiple domains. We excluded several domains with low-quality annotations, retaining a total of 36 domains. For our fine-tuning, we exclusively sampled data

[4] https://huggingface.co/models

from the training sets of these datasets to constitute our training data.

Hyperparameters We fine-tuned the LLaMA-2-13b-Chat checkpoint from Hugginface.[5]. We utilize Low Rank Approximation (LoRA) (Hu et al., 2021) and limited our fine-tuning to the parameters in the `q_proj` and `v_proj` modules. Further details about the fine-tuning hyperparameters can be found in Table 7. The fine-tuning was conducted on 4 A6000 48GB GPUs.

A.3 Supplementary Results

In this section, we present the detailed figures from the experimental results depicted in Table 3, Table 5, Figure 5, and Figure 6. Specifically, additional results from the end-to-end evaluation on MultiWOZ 2.2 are presented in Table 10 (complementing Table 3). Further results from the ablation studies with predicted and oracle functions are presented in Table 11 (complementing Table 5).

A.4 Details of Prompt Formatting

Conversation Context To format conversations within the prompt, we adopted the specific chat format for each LLM evaluated, as used in their respective instruction tuning steps.[6]

System prompt In our evaluation, we utilized the following system prompt template:

System prompt

You are a task-oriented assistant. You can use the given functions to fetch further data to help the users.

<FUNCTIONS>
{Function Specifications}
</FUNCTIONS>

<EXAMPLES>
{Example Conversations}
</EXAMPLES>

The parts surrounded in brackets and highlighted in blue serve as placeholders and are replaced with specific function specifications and example conversations related to that function/domain. The example part is only employed for few-shot prompting with the models not fine-tuned for function-calling.

Function Specifications For the function specification within the system prompt section of the prompt, we adhere to ChatGPT's format. To enhance model comprehension, we also experimented with translating the JSON format into a natural language description to include in the system prompt. An example illustrating both the JSON format and its corresponding natural language description for a specific domain is depicted in Figure 7.

Full Prompt Combining all components, an example of the full dialogue prompt is displayed in Figure 8. For clearer illustration, we adopt a more human-readable dialogue format not including the special tokens used in model-specific dialogue formats.

[5] https://huggingface.co/meta-LLaMA/LLaMA-2-13b-chat-hf

[6] https://github.com/lm-sys/FastChat

Model	Model versioning/path
GPT-3.5-Turbo	gpt-3.5-turbo-1106
GPT-4	gpt-4-1106-preview
Zephyr-7B-Beta	https://huggingface.co/HuggingFaceH4/zephyr-7b-beta
Vicuna-7B-v1.5	https://huggingface.co/lmsys/vicuna-7b-v1.5
Vicuna-13B-v1.5	https://huggingface.co/lmsys/vicuna-13b-v1.5
Baichuan2-13B-Chat	https://huggingface.co/baichuan-inc/Baichuan2-13B-Chat
LLaMA2-7B-Chat	https://huggingface.co/meta-llama/Llama-2-7b-chat-hf
LLaMA2-13B-Chat	https://huggingface.co/meta-llama/Llama-2-13b-chat-hf

Table 8: Evaluated LLMs in our experiments with their versions or Huggingface model paths.

Dataset	Domains	#Domains
Schema-Guided (Rastogi et al., 2020)	RentalCars_1, RentalCars_2, Buses_1, Buses_2, Events_1, Events_2, Services_1, Services_2, Services_3, Media_1, RideSharing_1, RideSharing_2, Travel_1, Hotels_1, Hotels_2, Hotels_3, Flights_1, Flights_2, Restaurants_1, Calendar_1, Music_1, Music_2, Weather_1, Movies_1, Homes_1, Banks_1	26
CamRest676 (Wen et al., 2016b)	Restaurant	1
MSR-E2E (Li et al., 2018)	Restaurant, Movie, Taxi	3
TaskMaster (Byrne et al., 2019)	pizza_ordering, movie_ticket, auto_repair, uber_lyft, coffee_ordering	5
WOZ (Mrkšić et al., 2016)	Restaurant	1

Table 9: Overview of the multi-turn TOD corpora utilized for fine-tuning, comprising a total of 36 diverse domains. This table details the datasets along with their specific domains and the number of domains included in each dataset.

JSON

```
{
  "name": "find_book_hotel",
  "description": "hotel reservations and vacation stays. ",
  "arguments": [
    {
      "name": "name",
      "type": "string",
      "description": "name of the hotel"
    },
    {
      "name": "pricerange",
      "type": "string",
      "description": "price budget of the hotel",
      "possible_values": ["expensive", "cheap", "moderate"]
    },
    ......
  ]
}
```

Natural Language Description (Text)

Function **name**: find_book_hotel
Function **description**: hotel reservations and vacation stays.
Function **arguments**:
- **name** (string): name of the hotel
- **pricerange** (string): price budget of the hotel (must be one of expensive, cheap, moderate)
- **people** (integer): number of people for the hotel booking (must be one of 1, 2, 3, 4, 5, 6, 7, 8)
- **stay** (integer): length of stay at the hotel (must be one of 1, 2, 3, 4, 5, 6, 7, 8)
- **stars** (integer): star rating of the hotel (must be one of 0, 1, 2, 3, 4, 5)
- **internet** (boolean): whether the hotel has internet (must be one of free, no, yes)
- **area** (string): area or place of the hotel (must be one of centre, east, north, south, west)

.....

Figure 7: The JSON format (left) and its corresponding natural language description (right) utilized in our evaluation. We take the hotel domain as an example.

Model	Attraction		Hotel		Restaurant		Taxi		Train		JGA		NLG	
	JGA	F1	JGA	F1	JGA	F1	JGA	F1	JGA	F1	Average	Overall	Inform	Success
ChatGPT (Hudeček and Dušek, 2023)	-	-	-	-	-	-	-	-	-	-	-	27.00	-	44.00
FNCTOD-LLAMA2-13B	62.43	85.55	**46.49**	**84.92**	61.51	89.36	69.11	81.13	**61.77**	**90.73**	**60.26**	**37.94**	**85.10**	44.50
ZEPHYR-7B-BETA	56.27	81.83	38.74	79.64	**62.91**	**91.16**	**74.03**	**86.26**	56.76	90.15	57.74	32.33	74.40	**57.40**
VICUNA-7B-V1.5	50.66	75.01	35.28	73.69	52.91	85.49	67.28	80.65	59.42	88.80	53.11	29.48	66.70	37.70
VICUNA-13B-V1.5	54.51	81.16	42.24	82.09	56.50	87.29	70.75	84.28	60.69	89.38	56.94	33.87	62.30	23.10
BAICHUAN2-13B-CHAT	53.13	79.98	40.43	81.78	58.90	87.84	69.11	82.24	60.69	89.37	56.45	33.02	67.70	45.70
LLAMA2-7B-CHAT	43.05	70.41	30.44	69.61	37.48	78.79	61.77	73.11	43.74	82.17	43.30	16.68	63.60	24.90
LLAMA2-13B-CHAT	49.95	76.91	29.53	67.58	48.64	81.44	64.72	68.90	53.64	85.02	49.30	25.83	64.20	27.70

Table 10: **Detailed end-to-end evaluation results on MultiWOZ 2.2**, complementing Table 3. The best performances in each column are in **bold**.

Model	Oracle Domain	FS Acc.	Attraction		Hotel		Restaurant		Taxi		Train		JGA	
			JGA	F1	JGA	F1	JGA	F1	JGA	F1	JGA	F1	Avg.	Overall
ChatGPT (GPT-4)	×	88.62	58.77	81.84	45.15	85.07	63.18	91.06	76.39	87.73	69.48	90.16	62.59	38.71
ChatGPT (GPT-4)	✓	-	65.61	87.42	49.08	87.07	64.54	90.99	77.97	88.53	72.12	93.02	65.86	44.70
ChatGPT (GPT-3.5)	×	95.54	67.15	87.20	37.56	82.86	60.12	90.21	74.43	86.90	67.29	92.48	61.31	38.56
ChatGPT (GPT-3.5)	✓	-	66.38	86.97	37.03	82.60	60.98	90.12	75.28	86.98	64.52	91.85	60.84	38.32
FNCTOD-LLAMA2-13B	×	91.68	62.24	84.99	46.83	85.39	60.27	88.69	67.48	80.39	60.90	89.88	59.54	37.67
FNCTOD-LLAMA2-13B	✓	-	62.88	85.91	47.55	85.33	59.67	88.61	71.02	82.36	61.61	90.25	60.55	38.70
ZEPHYR-7B-BETA	×	92.77	56.50	81.97	38.43	79.52	63.18	91.19	74.10	86.56	56.20	90.00	57.68	32.11
ZEPHYR-7B-BETA	✓	-	57.78	83.05	42.09	83.91	62.46	90.39	78.23	88.18	56.20	89.93	59.35	34.40
VICUNA-7B-V1.5	×	94.75	50.66	74.93	35.00	73.66	52.76	85.25	67.02	80.36	59.66	89.05	53.02	29.45
VICUNA-7B-V1.5	✓	-	50.59	74.80	36.00	74.38	52.58	84.75	69.11	81.81	59.13	88.39	53.48	30.06
VICUNA-13B-V1.5	×	91.82	54.25	80.99	38.43	79.96	56.44	87.26	69.11	83.37	58.82	89.26	55.41	31.84
VICUNA-13B-V1.5	✓	-	54.80	81.28	42.24	81.99	54.93	86.55	71.41	84.92	60.69	88.96	56.81	34.20
BAICHUAN2-13B-CHAT	×	92.50	53.67	79.57	40.15	81.36	59.02	87.82	69.31	81.95	60.67	89.45	56.56	33.21
BAICHUAN2-13B-CHAT	✓	-	56.43	80.62	40.34	82.55	59.88	88.95	69.84	82.64	61.35	89.23	56.97	34.94
LLAMA2-7B-CHAT	×	91.90	42.64	70.18	30.47	69.37	37.60	78.63	63.20	73.80	44.17	82.18	43.44	16.78
LLAMA2-7B-CHAT	✓	-	45.36	72.92	30.78	69.89	38.43	79.29	65.05	75.85	43.40	81.65	44.60	18.75
LLAMA2-13B-CHAT	×	89.34	49.76	76.80	29.50	67.60	48.87	81.33	64.66	68.97	53.59	85.09	49.28	25.68
LLAMA2-13B-CHAT	✓	-	50.40	77.59	30.28	68.91	49.08	81.09	65.97	69.70	54.80	84.89	50.11	29.56

Table 11: Detailed results regarding model performances with oracle and predicted domains/functions, complementing Table 5. FS Acc. indicates the function selection accuracy, and the JGA here indicates overall JGA.

The Full Dialogue Prompt

You are a task-oriented assistant. You can use the given functions to fetch further data to help the users.

```
<FUNCTIONS>
<FUNCTION>
{
  "name": "find_attraction",
  "description": "find touristy stuff to do around you.
  "arguments": [
    {
      "name": "area",
      "type": "string",
      "description": "area to search for attractions",
      "possible_values": ["centre", "east", "north", "south", "west"]
    },
    {
      "name": "name",
      "type": "string",
      "description": "name of the attraction"
    },
    ......
  ]
}
</FUNCTION>
<FUNCTION>
{
  "name": "find_book_hotel",
  "description": "hotel reservations and vacation stays,
  "arguments": [
    {
      "name": "name",
      "type": "string",
      "description": "name of the hotel"
    },
    {
      "name": "pricerange",
      "type": "string",
      "description": "price budget of the hotel",
      "possible_values": ["expensive", "cheap", "moderate”]
    },
    {
      "name": "parking",
      "type": "boolean",
      "description": "whether the hotel has parking",
      "possible_values": ["free", "no", "yes”]
    },
    ......
  ]
}
</FUNCTION>
</FUNCTIONS>
```

To call a function with a JSON object of the following format: {"function": "function_name", "arguments": {"argument1": "argument_value", "argument2": "argument_value"}}

User: i would like information about museums in cambridge .
Assistant: <function_call> {"function": "find_attraction", "arguments": {"type": "museum"}} </function_call> there are 23 museums in cambridge . what part of town will you be in ?
User: can you tell me about museums in the centre of town ?
Assistant: <function_call> {"function": "find_attraction", "arguments": {"type": "museum", "area": "centre"}} </function_call> i have the castle galleries museum in the centre of town . phone number is 01223307402 .
User: i also want to find the hotel called archway house .
Assistant: <function_call> {"function": "find_book_hotel", "arguments": {"name": "archway house"}} </function_call> it is located in the north , on 52 gilbert road , cb43pe . would you like to book a room ?
User: yes , please book for 4 nights on friday for 4 people .
Assistant: <function_call> {"function": "find_book_hotel", "arguments": {"name": "archway house", "area": "north", "people": "4", "day": "friday", "stay": "4"}} </function_call> all set , the reference number is 6s9gcfdo , can i help with anything else ?
User: that is all . thank you so much for your help .
Assistant: have a great day ! goodbye !

Figure 8: The complete dialogue format employed for model fine-tuning with no demonstration examples. Key components such as the function specifications in the system prompt, the function call , and the response in the assistant's output are highlighted for clarity. It's important to note that for easier comprehension, we've adopted a more human-readable dialogue format, and have simplified some function arguments for clearer visualization. In practice, we use the specific chat format tailored to each evaluated model, and the LLaMA2-Chat format is applied for fine-tuning the LLaMA2-Chat model.